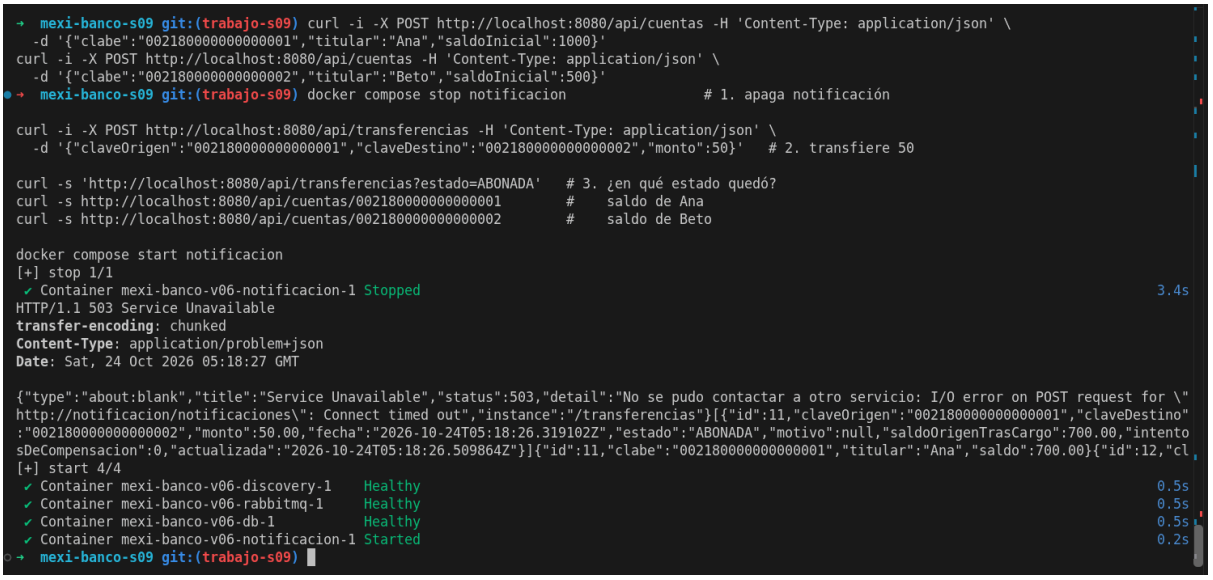
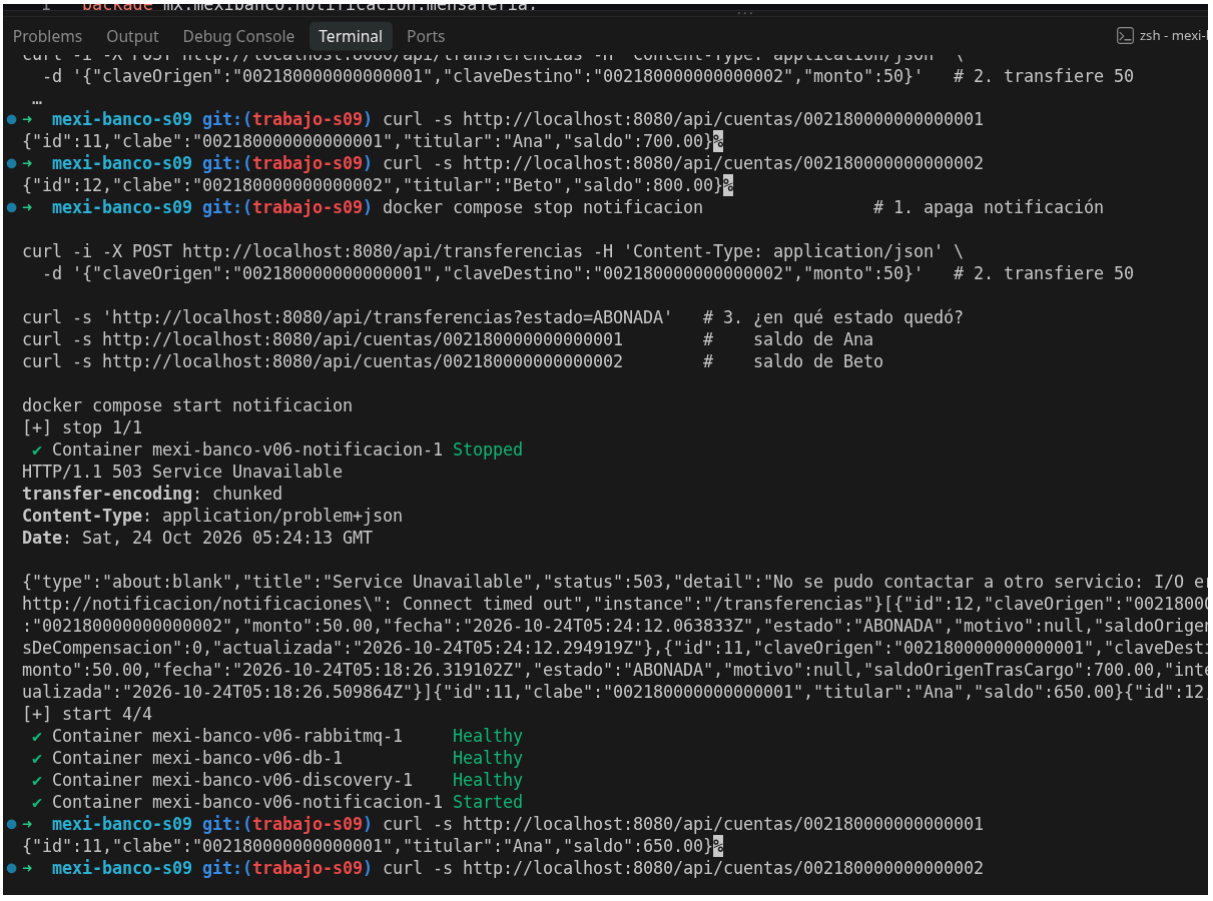
Erick Nava Santiago
Sistemas Distribuidos - LAB 09 Mexi Banco: que el aviso no tumbe la transferencia

1. Problema Inicial

Se comenzó con la descargar de la solución anterior



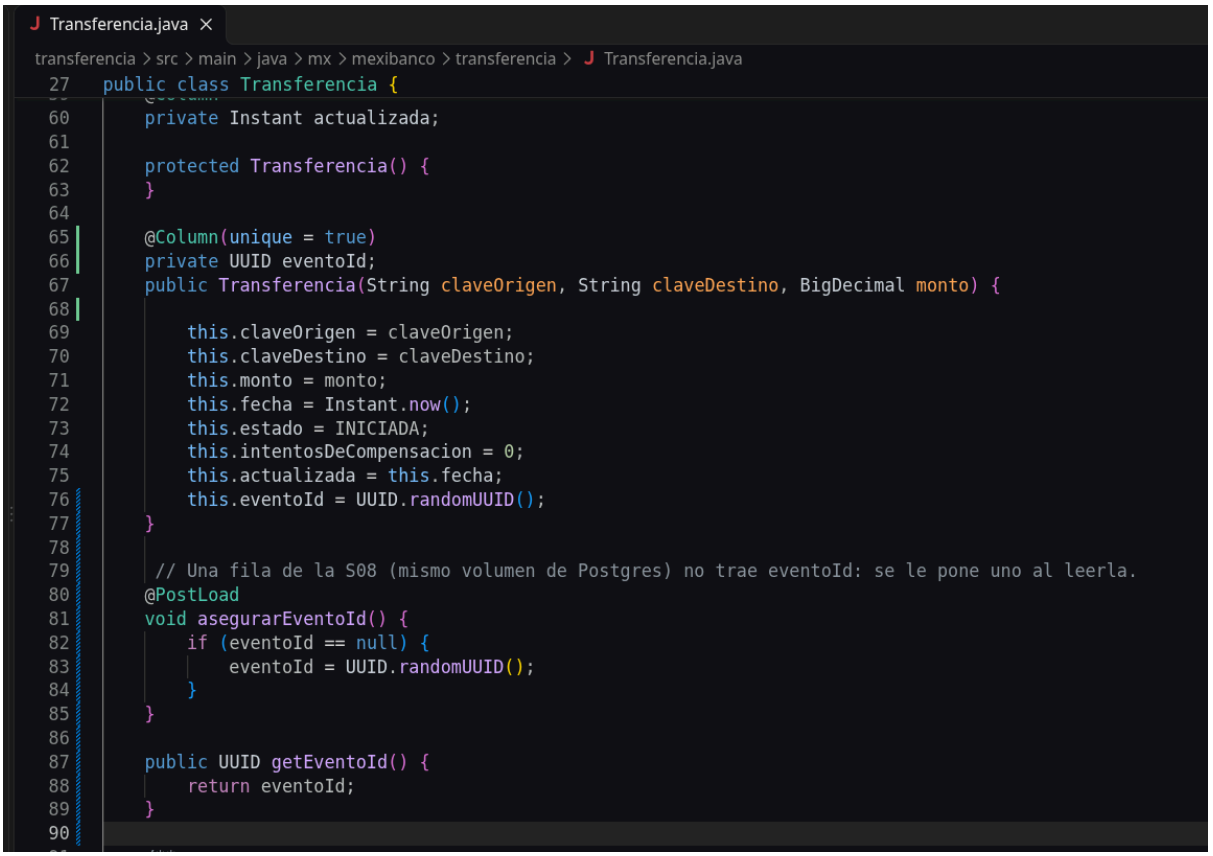
Se procedió con la creación de cuenta y la introducción de la transacción. Donde, efectivamente se logra obtener un intento de compensación.



En esta segunda captura se logra ver que los saldos sí lograron cambiar. El fallo se debe a la llamada síncrona de http, la cual esperaba la saga para poder enviar los datos.

2. Descripción de hecho y http

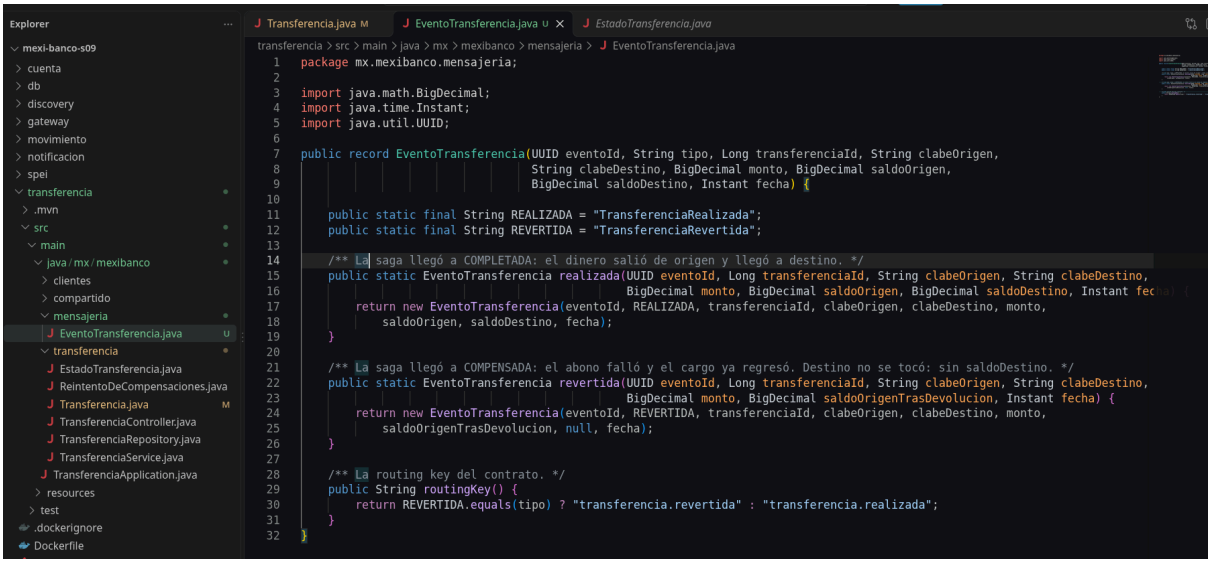
En transferencia.java se realizo el cambio siguiente:



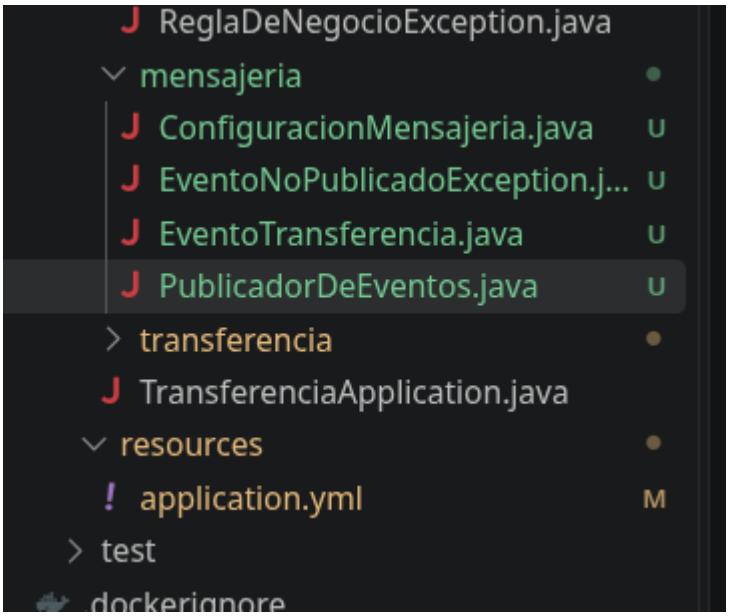
La cual, asigna un evento id al leer la transferencia.

Posteriormente se creó el paquete de mensajería. Donde se incluyó el contrato y la configuración que incluye el exchange y el convertidor a json:

Erick Nava Santiago
Sistemas Distribuidos - LAB 09 Mexi Banco: que el aviso no tumbe la transferencia

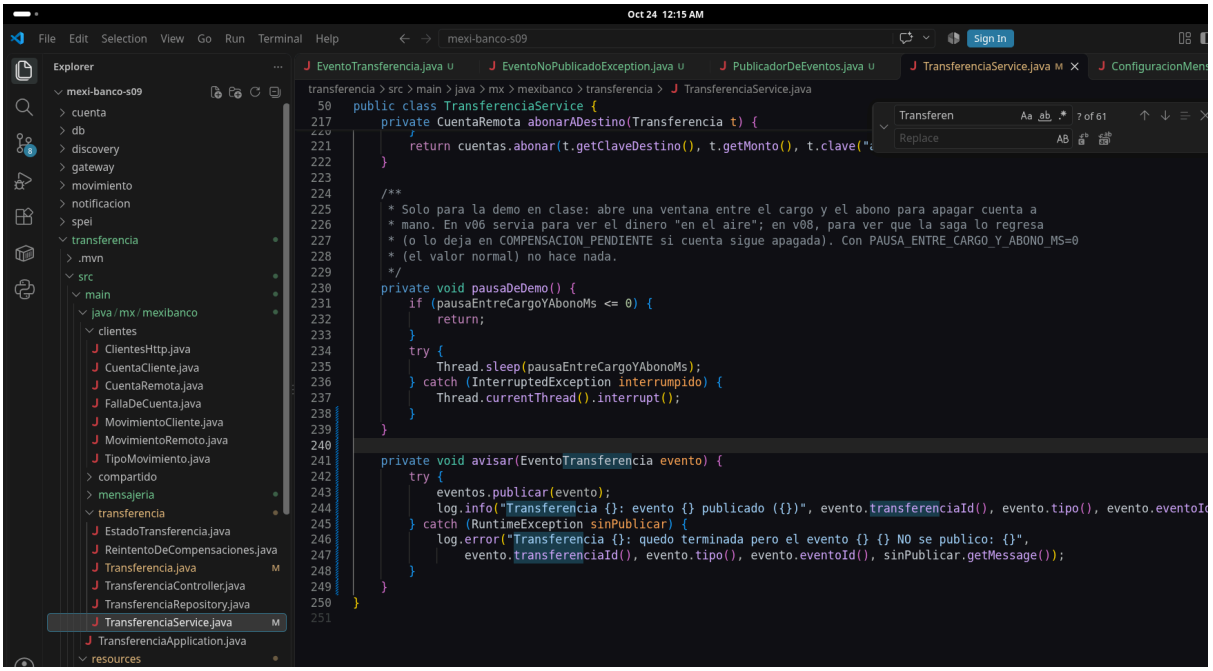


También se agregó una excepción en caso de fallo y el publisher para mensajería



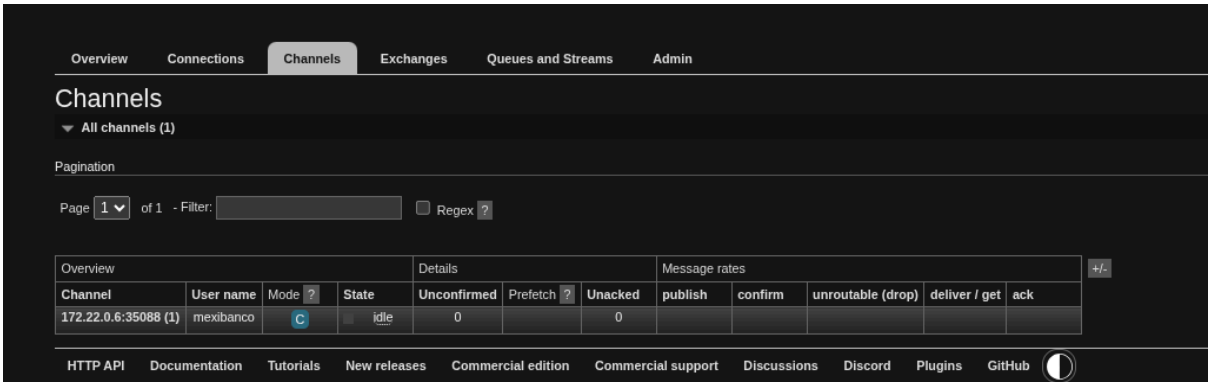
Se eliminó cliente/notificacioncliente.java y se modifico el transferencaiservice.java que se encargara de orquestar la transferencia interna.

Erick Nava Santiago
Sistemas Distribuidos - LAB 09 Mexi Banco: que el aviso no tumbe la transferencia



Se ejecutó:





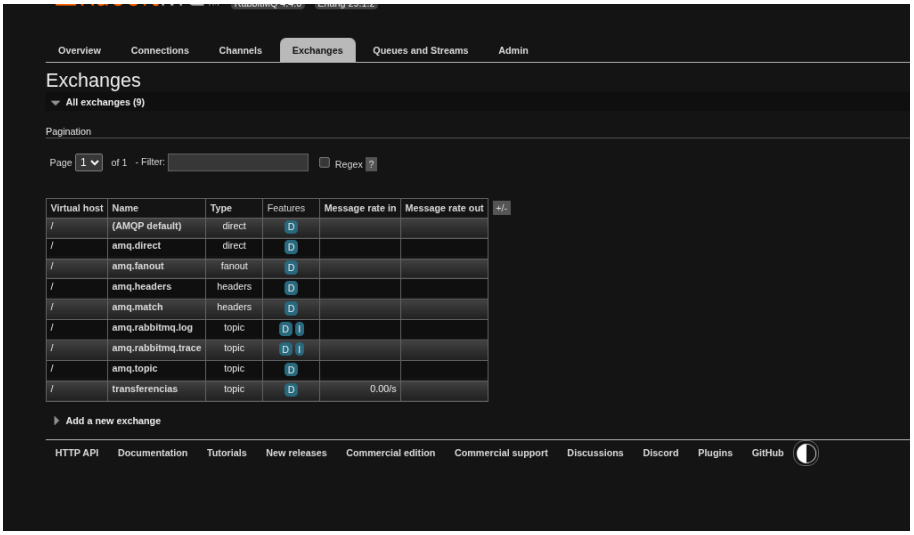
Al notar las inconsistencias se volvió a ejecutar y se pudo visualizar el 201

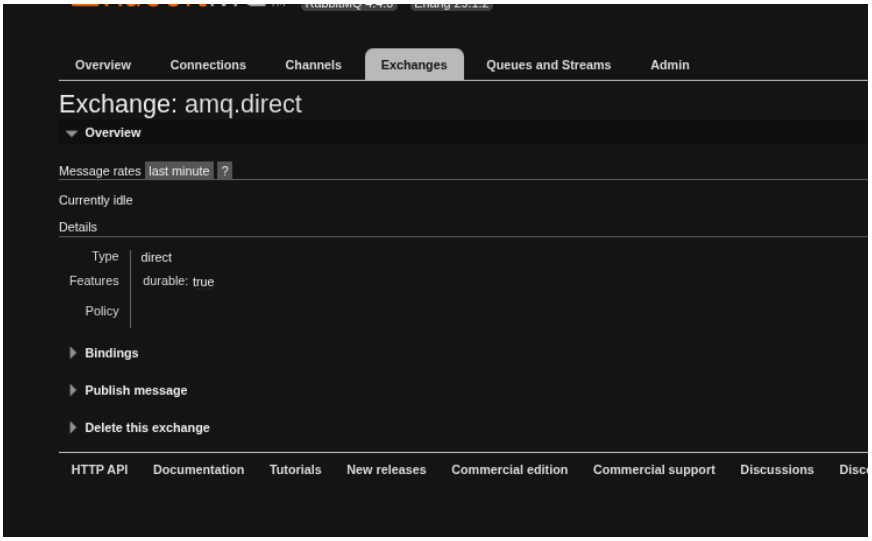
Erick Nava Santiago
Sistemas Distribuidos - LAB 09 Mexi Banco: que el aviso no tumbe la transferencia

```
➜  mexi-banco-s09 git:(trabajo-s09) ✗ docker compose up --build -d transferencia      # reconstruye solo transferencia
docker compose stop notificacion

curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50}' …
 ✔ Image mexi-banco-v06-discovery           Built                                                                    1.1s
 ✔ Container mexi-banco-v06-transferencia-1 Running                                                                  0.0s
 ✔ Container mexi-banco-v06-discovery-1     Healthy                                                                  0.5s
 ✔ Container mexi-banco-v06-db-1            Healthy                                                                  0.5s
 ✔ Container mexi-banco-v06-rabbitmq-1      Healthy                                                                  0.5s
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped                                                                   0.0s
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 06:36:44 GMT

{"id":13,"claveOrigen":"00218000000000001","claveDestino":"00218000000000002","monto":50,"fecha":"2026-10-24T06:36:43.536266601Z","estado":"COM
PLETADA","motivo":null,"saldoOrigenTrasCargo":600.00,"intentosDeCompensacion":0,"actualizada":"2026-10-24T06:36:44.759092826Z","eventoId":"4a66fc
47-08c3-44ce-af6b-28aaed1d4c96"}transferencia-1  | 2026-10-24T06:36:44.837Z ERROR 1 --- [transferencia] [nio-8080-exec-2] m.m.transferencia.Trans
ferenciaService    : Transferencia 13: quedo terminada pero el evento TransferenciaRealizada 4a66fc47-08c3-44ce-af6b-28aaed1d4c96 NO se publico: N
inguna cola recibio el evento 4a66fc47-08c3-44ce-af6b-28aaed1d4c96 con la routing key transferencia.realizada (NO_ROUTE)
➜  mexi-banco-s09 git:(trabajo-s09) ✗
```

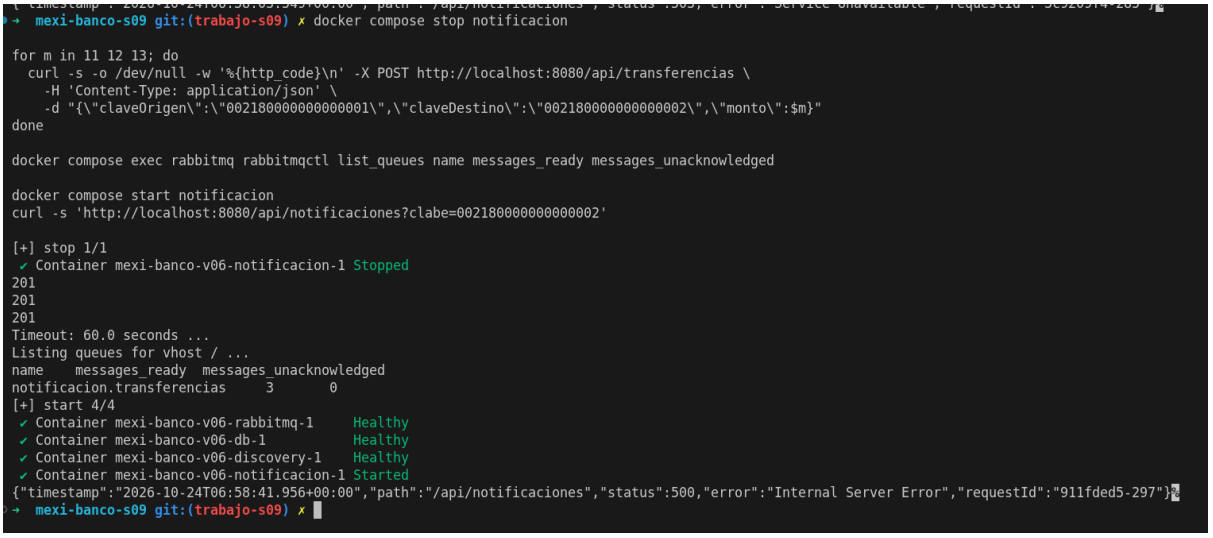


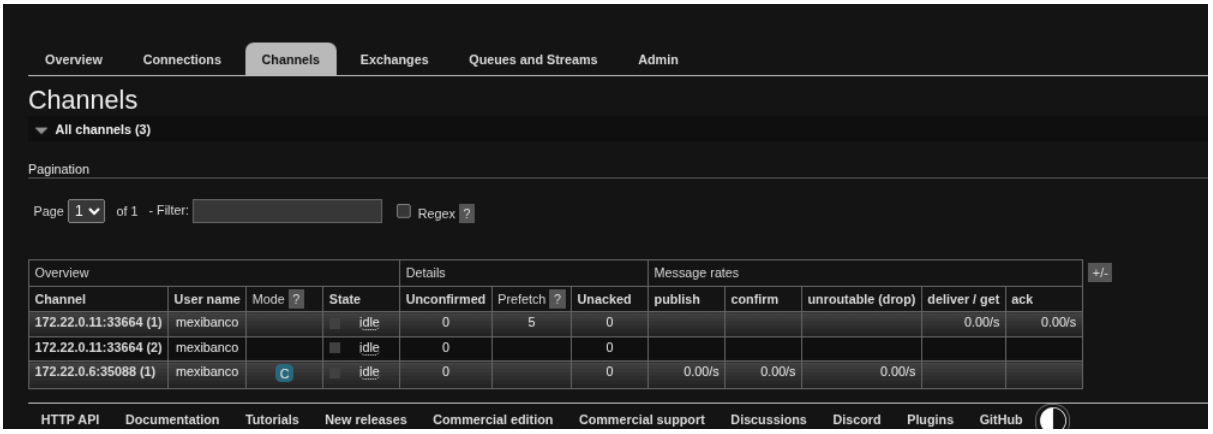


¿Qué garantiza la confirmación? Garantiza que el broker recibe el mensaje y se hizo cargo de él. El ack llega después de que rabbit lo guardo en disco. No garantiza que una cola lo haya recibido, que la notificación se haya leído o procesado o si quiera que el evento llegue a publicarse.

3. Se buscó agregar el servicio de notificación

Al ejecutar se logra ver que las tres transferencias se realizaron correctamente y se tuvieron sus mensajes listos.





y se logró ver con la prueba, donde se logra ver el aviso.



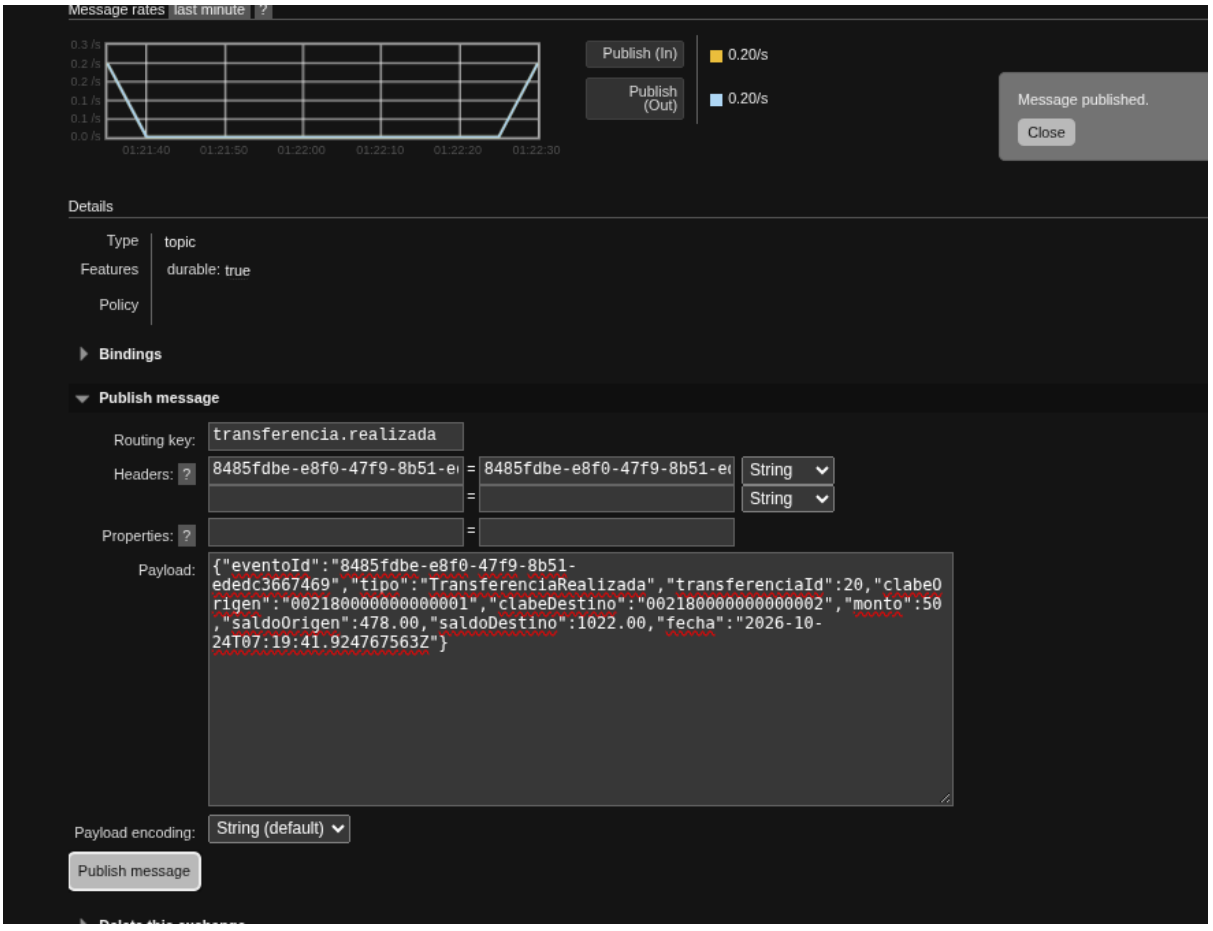
¿Por qué el ack va después de guardar y no antes? Esto es por que si se manda antes estaría borrando el mensaje del buzón antes de que se guarde en la base, se coloca

después para mantener la persistencia de los datos. El aviso en cada caso se repite, o sea, se reentrega

Tarea Duplicado:

Se ingresó a la sección mencionada y al visualizar:



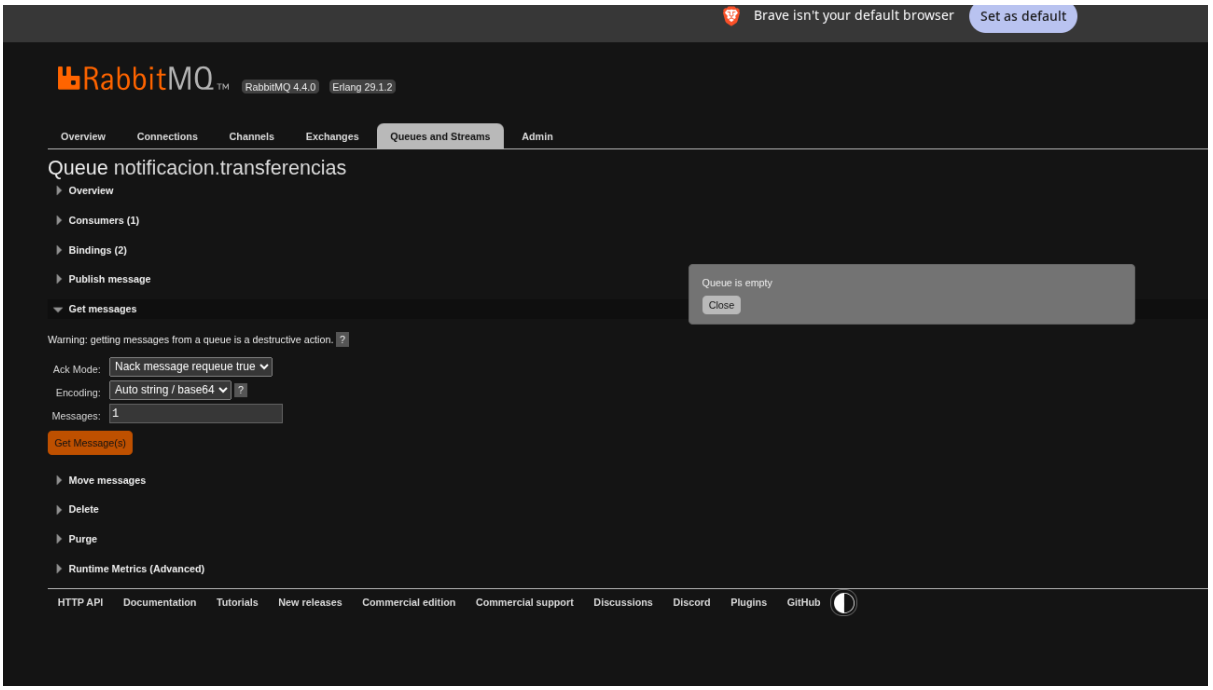


Y se pueden visualizar 2 listos

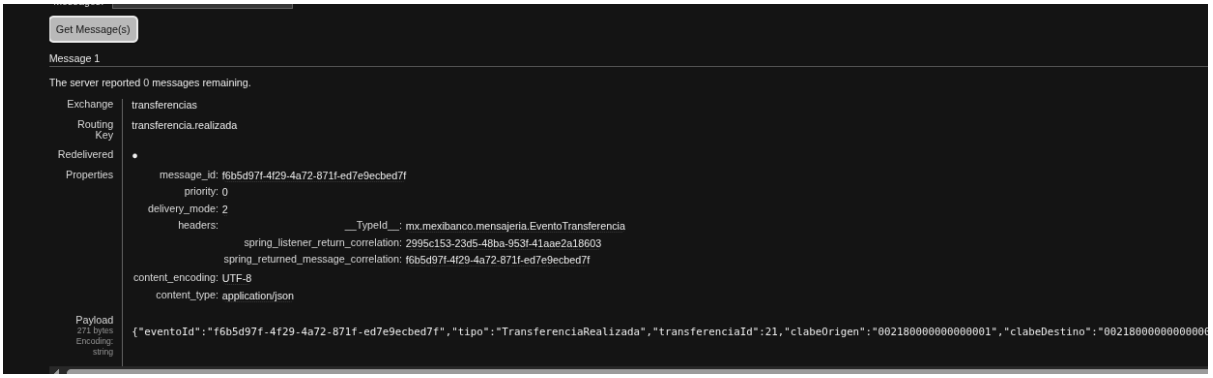
Erick Nava Santiago
Sistemas Distribuidos - LAB 09 Mexi Banco: que el aviso no tumbe la transferencia



Seguido de eso, get messenger nos devolvera que no hay mensajes

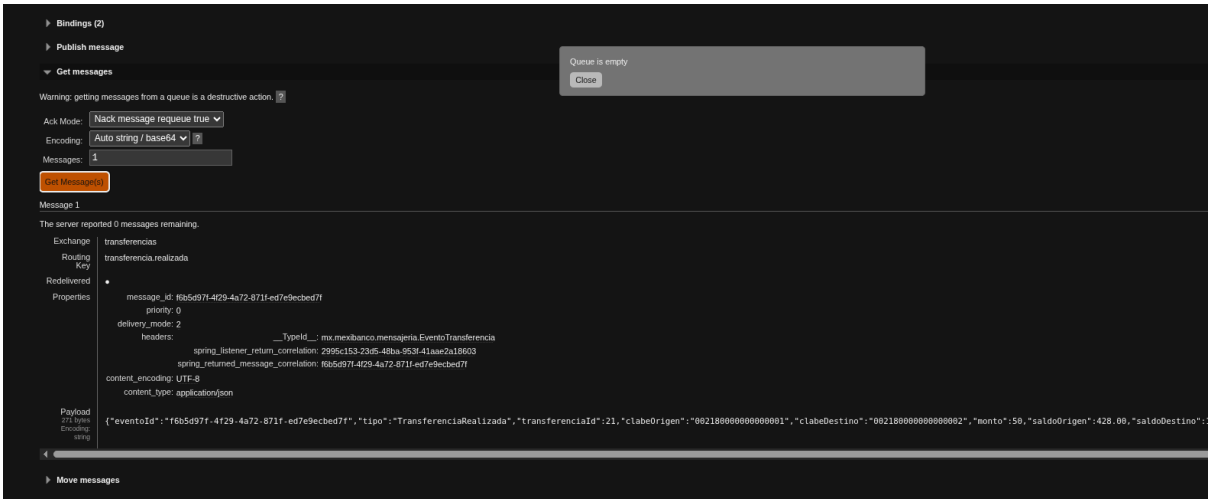


Haciendo el cambio en el descanso solicitado:

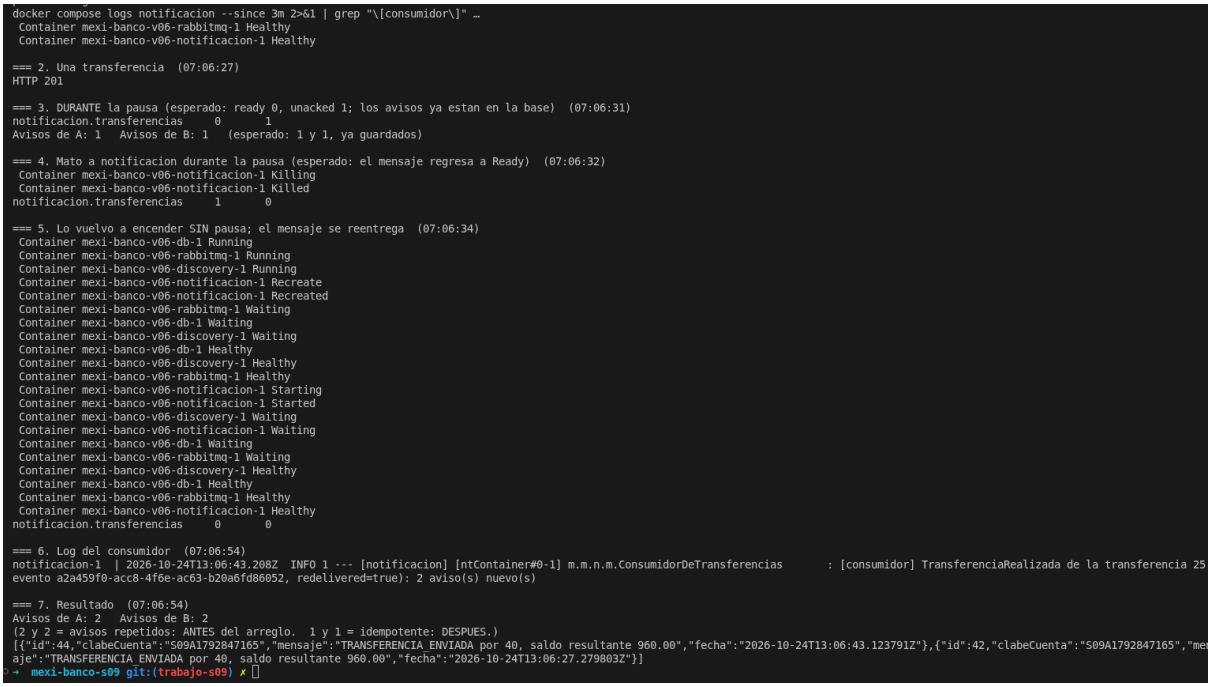


Entre el periodo solicitado se mostró en cola, pero cambió su estado recien paso la ventana de tiempo:

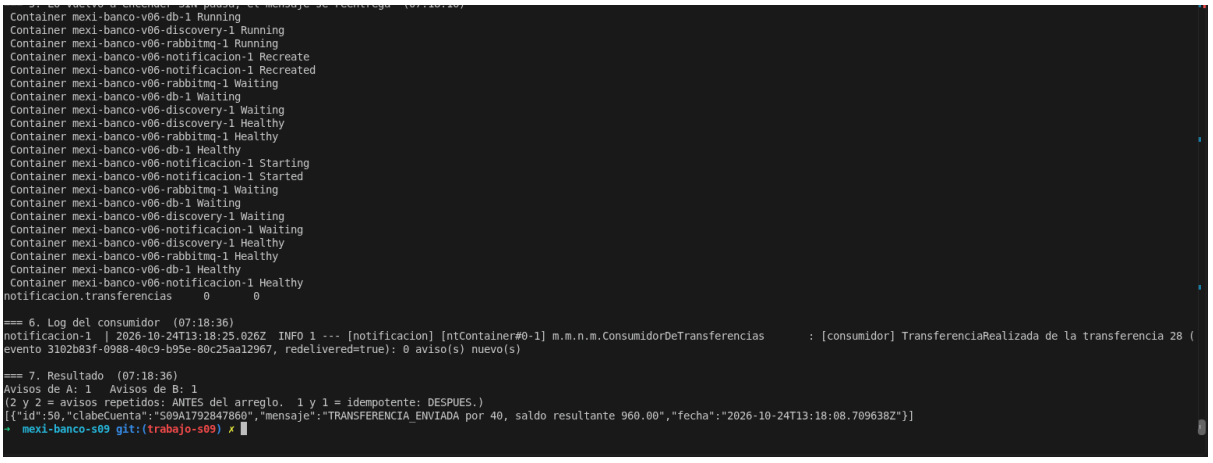
Erick Nava Santiago
Sistemas Distribuidos - LAB 09 Mexi Banco: que el aviso no tumbe la transferencia



Viendo los logs desde la terminal:



Corrigiendo:



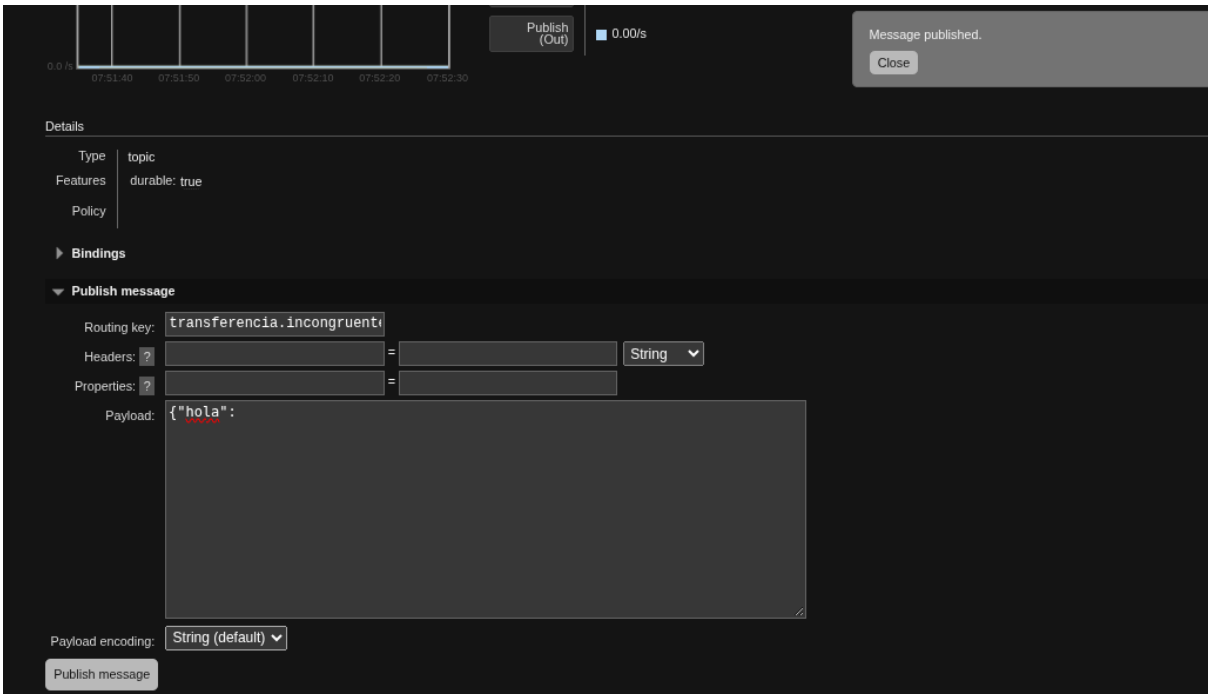
Se puede ver que al crear el idempotente no se reciben los avisos repetidos.

¿Por qué en la misma transacción? Porque si fueran dos transacciones separadas, al momento de recibir la primera se marcaría como completa, y después, al recibir la segunda, el valor ya estaría como realizado. Lo cual, terminaría siendo mal ya que se descarta el mensaje y se perdería.

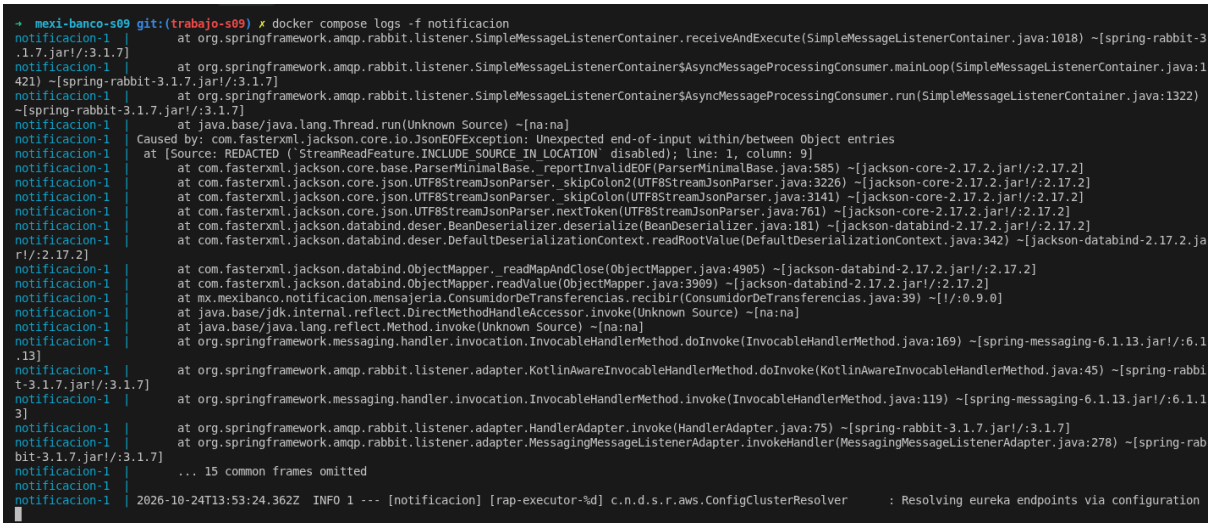
Se utiliza el on conflict do nothing para que no haya errores de concurrencia, pues si llegasen al mismo tiempo, se tendría un duplicado y un error en la consistencia de los datos. Al ser atómica la base de datos, decidirá cuál insertar y mantener la congruencia de la base.

Tarea dlq:

Para esto parte se creó el mensaje incongruente:

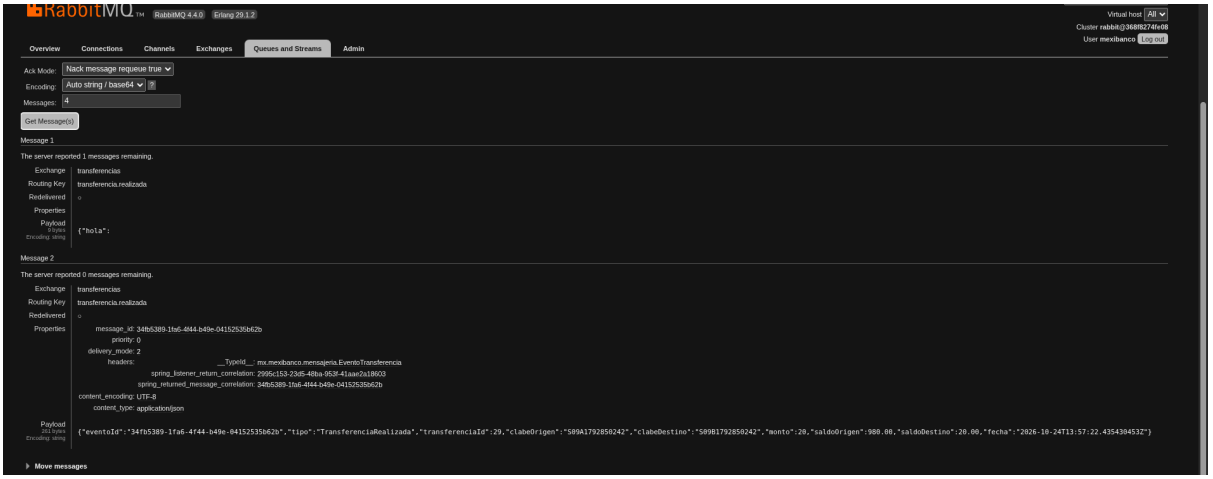


Y se dejó visualizando los logs

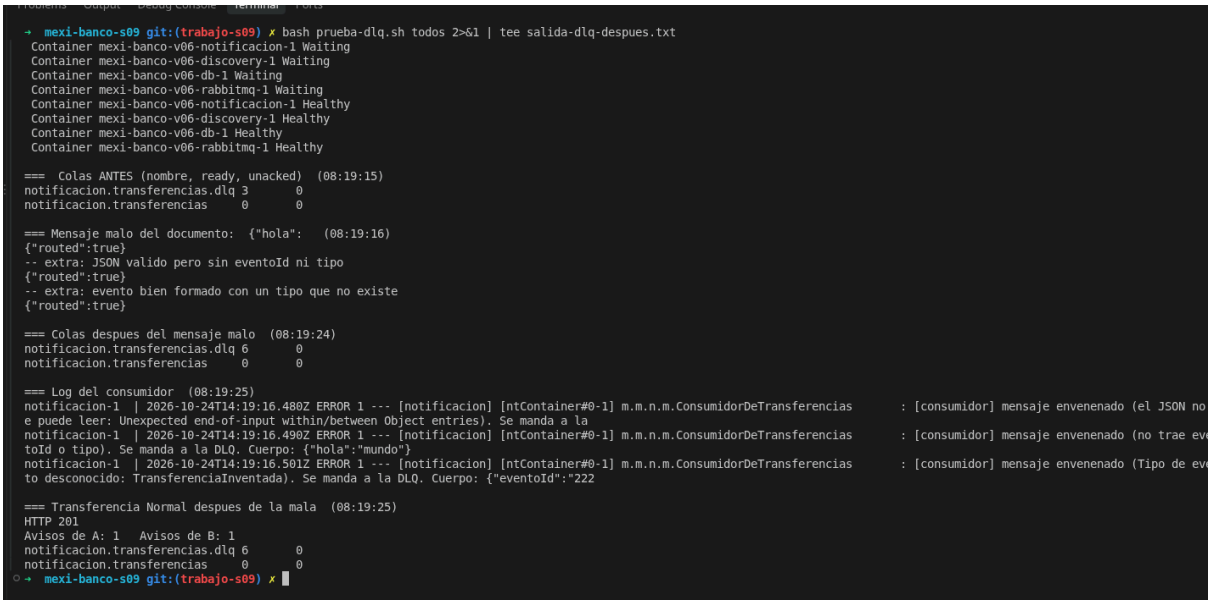


Erick Nava Santiago
Sistemas Distribuidos - LAB 09 Mexi Banco: que el aviso no tumbe la transferencia

Tras realizar una transferencia normal se logra ver que queda en lista



Al correr y hacer los cambios se ve que quedan en espera un mensaje



Preguntas:

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?
   Si llega mientras el sistema está caído, el evento de transferencia no puede ser publicado en la cola. El broker no responderá y se tendrá un fallo de sistema. No lo arregla el publisher por que no existe ack activo el cual pueda manejarlo

2. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?
   Para garantizar que el mensaje solo se dé por procesado (y se elimine de la cola) una vez que la base de datos ha persistido exitosamente el registro de la notificación. Se garantiza que si el servicio cae después de recibir el mensaje pero antes de guardarlo el mensaje se vuelva a encolar. Al revés, el mensaje se perdería.

3. Elige y di qué sacrificaste: al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste? Tres líneas: elección, porque, sacrifico.

Erick Nava Santiago
Sistemas Distribuidos - LAB 09 Mexi Banco: que el aviso no tumbe la transferencia

Elección: Desacople utilizando cola asíncrona con reintento DLQ
Porque: Aísla el flujo evitando fallos, caídas o bloqueos en operaciones críticas.
Sacrificio: Se pierde la consistencia inmediata y se incrementa la complejidad